AutoDD: Learning to Select Drift Detectors for Evolving Data Streams

Anonymous

Abstract

Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, selecting and configuring the most suitable detector for a specific data stream remains a challenging task. Existing AutoML frameworks for streaming data automate model selection rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces **AutoDD** (Automated Drift Detector), a novel framework that jointly automates detector selection and hyperparameter configuration using a hierarchical learning approach. AutoDD extracts comprehensive online meta-features and employs a two-level hierarchical selection algorithm to adaptively choose both the detector and its configuration. Theoretical analysis provides sublinear regret bounds ensuring convergence to optimal performance. Comprehensive experiments on synthetic and real-world datasets demonstrate that AutoDD consistently outperforms state-of-the-art fixed detectors and learning-based alternatives while maintaining computational efficiency.

1 Introduction

Concept drift, where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [ao Gama *et al.*, 2014], incremental learning [Losing *et al.*, 2018], and non-stationary learning [Lu *et al.*, 2019]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. Concept drift manifests in various forms, including sudden (abrupt) drift, gradual drift, incremental drift, and recurring drift. Figure 1 illustrates these distinct types of concept drift.

Over the past two decades, numerous drift detection methods have been developed to address this challenge, including error rate-based approaches such as DDM [Gama *et al.*, 2004], ADWIN [Bifet and Gavaldà, 2007], and ADDM [Mayaki and Riveill, 2022], distribution-based methods like KSWIN [Raab *et al.*, 2020], uncertainty-based methods such as PUDD [Lu *et al.*, 2025], and meta-learning techniques like Meta-ADD [Yu *et al.*, 2021]. However, a fundamental challenge remains unresolved in systematically selecting and configuring the most appropriate drift detector for a given data stream.

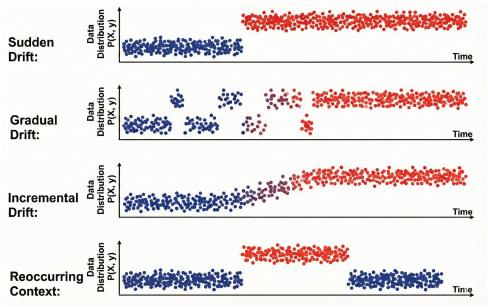


Figure 1: Types of concept drifts.

Automated Machine Learning (AutoML) has revolutionized the field by addressing similar selection challenges in batch learning scenarios [Hutter *et al.*, 2019]. Recent advancements have extended AutoML to streaming data, enabling automatic selection and configuration of classifiers [Celik *et al.*, 2023; Verma *et al.*, 2024]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in the automation framework.

To bridge this gap, we introduce **AutoDD** (Automated Drift Detector), a modular framework that automates drift detector selection and configuration using hierarchical bandits, compatible with any classifier or AutoML pipeline.

- We propose a two-level hierarchical selection algorithm using LinUCB [Li *et al.*, 2010] and Thompson Sampling [Agrawal and Goyal, 2012] that adaptively selects detectors and their hyperparameters based on online meta-features.
- We formalize detector selection with a theoretical analysis that provides sublinear regret bounds guaranteeing convergence to optimal selections.
- Extensive evaluation demonstrates AutoDD outperforms state-of-the-art detectors while maintaining computational efficiency.

The rest of this paper is organized as follows. Section 2 presents the related work. Section 3 provides the details of our proposed AutoDD framework. Section 4 describes the experimental setup. Section 5 presents the experimental results and analysis. Finally, Section 6 concludes the paper.

2 Related Work

Drift detection methods can be broadly categorized into three paradigms. **Error rate-based methods** monitor fluctuations in classifier error rates, triggering alarms when changes are statistically significant. DDM [Gama *et al.*, 2004] pioneered the warning-drift framework by monitoring prediction accuracy under binomial distribution assumptions, followed by EDDM [Baena-García *et al.*, 2006] which focuses on distances between errors to detect gradual drift, HDDM [Friás-Blanco *et al.*, 2015] which applies Hoeffding's inequality with exponentially weighted moving averages, and ADWIN [Bifet and Gavaldà, 2007] which maintains adaptive sliding windows that automatically shrink when distribution changes are detected. Page-Hinkley [Page, 1954] applies cumulative sum control charts to detect mean shifts in sequential data. Recent advances include MDDM [Pesaranghader *et al.*, 2017], which uses McDiarmid's inequality for shorter detection delays with tighter concentration bounds, and ADDM [Mayaki and Riveill, 2022], which models error rates as SETAR time series [Tsay, 1989] to achieve lower false alarm rates. **Distribution-based methods** compare data distributions across windows by quantifying dissimilarity between new and historical data distributions [Kifer *et al.*, 2004], including approaches based on information theory [Dasu *et al.*, 2006], competence models [Lu *et al.*, 2014], and the Kolmogorov-Smirnov test (KSWIN) [Raab *et al.*, 2020] which detects changes in univariate feature distributions. **Uncertainty and learning-based methods** address limitations of error-rate monitoring: PUDD [Lu *et al.*, 2025] leverages prediction confidence for earlier detection by monitoring changes in prediction uncertainty distributions, while Meta-ADD [Yu *et al.*, 2021] employs prototypical networks to classify drift types (sudden, gradual, incremental) via meta-learning on diverse drift scenarios.

Recent developments in automated algorithm selection and hyperparameter optimization (CASH) [Hutter *et al.*, 2019] have been extended to streaming scenarios [**?**]. However, all existing streaming AutoML systems assume a pre-selected drift detector, requiring manual specification and leaving a critical gap in the automation framework. Despite advances in both drift detection and streaming AutoML, drift detector selection remains manual. Our work fills this gap by extending AutoML to automatic drift detector selection and configuration.

3 Automated Drift Detector

In this section, we elaborate on Automated Drift Detector (AutoDD). As illustrated in Figure 2, AutoDD employs a closed-loop architecture with the following main steps. First, the system initializes the detector portfolio with standard hyperparameters and sets up the contextual bandit parameters. Then, during online operation, meta-features characterizing stream velocity, stationarity, and temporal patterns are continuously extracted from the incoming data stream. These meta-features serve as context for the LinUCB agent, which selects the most appropriate detector from the portfolio. Subsequently, Thompson Sampling configures the selected detector's hyperparameters for optimal performance. The configured detector monitors the stream and triggers model adaptation upon detecting drift. Finally, performance feedback is computed based on detection outcomes and accuracy changes, which updates both the LinUCB and Thompson Sampling components to refine future selections. This adaptive learning cycle enables AutoDD to continuously improve its detector-stream matching over time.

3.1 Algorithm Overview

AutoDD operates over sliding windows of size w, where w denotes the global AutoDD window size used throughout this paper. When each window fills with incoming instances, the framework executes a complete selection-detection-update cycle. This cycle begins with extracting stream meta-features.

Meta-Feature Extraction: We design 15 meta-features organized into three categories - Statistical, Stationarity, and Temporal. Table 1 summarizes each meta-feature with its mathematical formulation. All features are computed over a sliding window of size w using online statistics and normalized to $[0, 1]$ using running min-max statistics to ensure scale consistency. Let Δt denote the elapsed time (or instance count for index-based streams) within the current window. The context vector $\boldsymbol{\phi}_t \in \mathbb{R}^{15}$ is computed incrementally, ensuring $O(1)$ update complexity per instance.

Table 1: Meta-feature portfolio with mathematical formulations

Category	Feature	Formula/Description
Statistical	Velocity	$v_t = n_t/\Delta t$
	Dimensionality	d
	Sparsity	$\frac{1}{wd}\sum_{i=1}^{w}\sum_{j=1}^{d}\mathbb{I}[\mathbf{x}_{i,j}=0]$
	Noise level	$H(\mathbf{X}) \approx -\frac{1}{w}\sum_{i=1}^{w}\log\hat{p}_k(\mathbf{x}_i)$ (k-NN, $k=5$)
	Class imbalance	$\max_c \lvert C_c\rvert / \min_c \lvert C_c\rvert$
Stationarity	Error volatility	$\sigma_{err}(t) = \sqrt{\frac{1}{w}\sum_{i=t-w}^{t}(e_i-\bar{e})^2}$
	KL divergence	$D_{KL}(P_t \Vert P_{t-1})$ (histogram, $\sqrt{w}$ bins)
	Autocorrelation	$\rho_t = \frac{\mathrm{Cov}(e_t, e_{t-1})}{\sigma_{e_t}\sigma_{e_{t-1}}}$
	Mean shift	$\Delta\mu_t = \frac{1}{d}\sum_{j=1}^{d}\lvert\mu_{t,j}-\mu_{t-1,j}\rvert$
	Variance shift	$\Delta\sigma_t^2 = \frac{1}{d}\sum_{j=1}^{d}\lvert\sigma_{t,j}^2-\sigma_{t-1,j}^2\rvert$
Temporal	Trend	$\beta_{\text{trend}} = \frac{\mathrm{Cov}(t, e_t)}{\mathrm{Var}(t)}$
	Periodicity	$f_{\text{dom}} = \arg\max_f \lvert\mathcal{F}\{e_t\}(f)\rvert$
	Change frequency	$\frac{1}{\Delta t}\sum_{i=1}^{w}\mathbb{I}[\lvert\Delta e_i\rvert > \tau]$
	Change magnitude	$\frac{1}{\lvert\mathcal{C}_t\rvert}\sum_{i\in\mathcal{C}_t}\lvert\Delta e_i\rvert$
	Recurrence score	$\frac{1}{w^2}\sum_{i,j}\mathbb{I}[\lVert\mathbf{x}_i-\mathbf{x}_j\rVert < \epsilon]$

Detector Portfolio: AutoDD maintains a diverse portfolio of K drift detectors spanning statistical, error-based, and distribution-based paradigms. Table 2 summarizes each detector with its hyperparameters, valid ranges, sampling scales, and default values from the literature. For each detector i, we maintain C hyperparameter configurations denoted $\boldsymbol{\eta}_{i,c}$ for $c \in [C]$: the first is always the literature default, while the remaining $C-1$ are sampled from the specified ranges using the appropriate scale (logarithmic, linear, or integer).

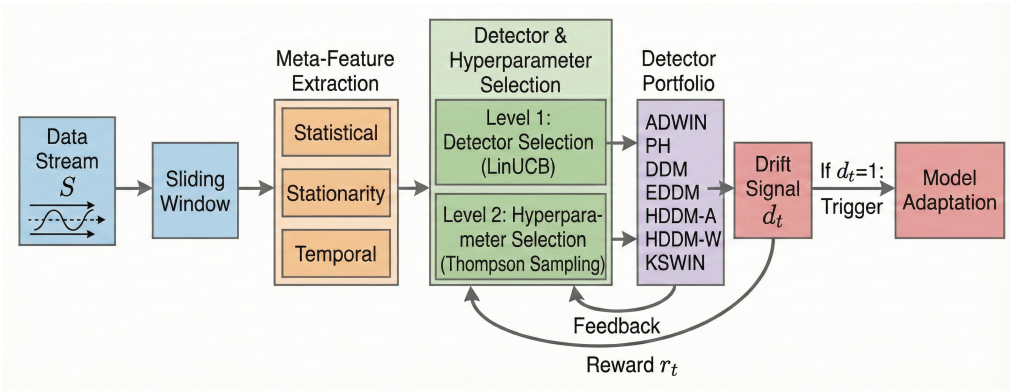


Figure 2: The AutoDD framework consisting of meta-feature extraction, LinUCB-based selection agent, and detector portfolio. The agent adaptively selects detectors based on stream characteristics and updates its policy through performance feedback.

Table 2: Detector portfolio with hyperparameter configurations

Detector	Type	Parameter	Range	Scale	Default
ADWIN	Statistical	δ_{adwin}	[0.0001, 0.1]	log	0.002
Page-Hinkley	Statistical	θ_{ph} δ_{ph}	[10, 100] [0.001, 0.01]	int log	50 0.005
DDM	Error-based	w_{thresh} d_{thresh}	[1.5, 3.0] [2.0, 4.0]	linear linear	2.0 3.0
EDDM	Error-based	α_{eddm} β_{eddm}	[0.90, 0.99] [0.85, 0.95]	linear linear	0.95 0.90
HDDM-A	Error-based	δ_{hddm}	[0.0001, 0.01]	log	0.001
HDDM-W	Error-based	λ_{hddm}	[0.01, 0.1]	log	0.05
KSWIN	Distribution	α_{ks} w_{ks}	[0.001, 0.05] [50, 500]	log int	0.005 100

3.2 Hierarchical Selection Method

AutoDD uses a two-level hierarchical selection strategy that jointly optimizes detector choice and hyperparameter configuration while maintaining computational efficiency. This approach avoids the combinatorial explosion of $K \times C$ configurations by decomposing the problem into sequential decisions.

Level 1 - Detector Selection via LinUCB: We formulate detector selection as a contextual multi-armed bandit problem where context is $\boldsymbol{\phi}_t$ (meta-features), arms are K detectors, and reward is detection performance. LinUCB [Li *et al.*, 2010] assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i + \epsilon_t \tag{1}$$

where $\boldsymbol{\theta}_i \in \mathbb{R}^m$ is the unknown parameter vector for detector i, with m being the meta-feature dimensionality (i.e., $m = 15$). The algorithm selects the detector with highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \tag{2}$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1} \mathbf{b}_i$ is the ridge regression estimate. Here, $\mathbf{A}_i = \gamma \mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s \boldsymbol{\phi}_s^\top \in \mathbb{R}^{m \times m}$ is the regularized design matrix with regularization parameter $\gamma > 0$, and $\mathbf{b}_i = \sum_{s:i_s=i} r_s \boldsymbol{\phi}_s \in \mathbb{R}^m$ is the cumulative reward-weighted context vector for detector i. The parameter $\alpha_{\text{ucb}} > 0$ controls exploration-exploitation balance.

Level 2 - Hyperparameter Selection via Thompson Sampling: Given the selected detector D_{i_t}, we adaptively configure its hyperparameters using Thompson Sampling [Agrawal and Goyal, 2012]. For each configuration $c \in [C] = \{1, \ldots, C\}$ of detector i, we maintain Beta distribution parameters $(a_{i,c}, b_{i,c})$ initialized to $(1, 1)$. At each window, we sample:

$$\psi_{i_t,c} \sim \text{Beta}(a_{i_t,c}, b_{i_t,c}) \quad \forall c \in [C] \tag{3}$$

and select configuration:

$$c_t = \arg\max_c \psi_{i_t,c} \tag{4}$$

After observing reward r_t, we update:

$$a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t \tag{5}$$

$$b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t) \tag{6}$$

where $\tilde{r}_t = (r_t + 1)/2$ normalizes the reward to $[0, 1]$.

3.3 Evaluation and Adaptation

Evaluating detector performance without ground truth drift labels is challenging. Following standard practice, we use model accuracy drop as a proxy metric. The reward at time t is:

$$r_t = \begin{cases} 0.5 + \min(\Delta a_t \cdot 2, 0.5) & d_t = 1, \Delta a_t > \tau \\ -0.3 & d_t = 1, \Delta a_t \leq \tau \\ -0.5 - \min(\Delta a_t, 0.5) & d_t = 0, \Delta a_t > \tau \\ 0.1 & d_t = 0, \Delta a_t \leq \tau \end{cases} \tag{7}$$

where $d_t \in \{0, 1\}$ indicates drift detection at time t, $\Delta a_t = a_{t-w} - a_t$ is the accuracy drop between consecutive windows, and $\tau = 0.05$ is the significance threshold. The reward function penalizes both false positives and false negatives while rewarding correct drift detections and stable periods.

Upon drift detection, the model undergoes complete reset. Let $\mathcal{M}_t$ denote the classifier state at time t. When drift is detected ($d_t = 1$), we perform:

$$\mathcal{M}_t \leftarrow \mathcal{M}_0 \tag{8}$$

Algorithm 1 AutoDD Algorithm

Input: Stream $\mathcal{S}$, Portfolio $\mathcal{D}$, Window w
Parameter: Exploration α_{ucb}, Regularization γ, Configs per detector C
Output: Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialize $\mathbf{A}_i \leftarrow \gamma \mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$
Initialize $a_{i,c} \leftarrow 1$, $b_{i,c} \leftarrow 1$ for all $i \in [K], c \in [C]$
Initialize classifier $\mathcal{M}_0$, sliding window $W \leftarrow \emptyset$
for each instance $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$
  if $|W| = w$ **then**
    Extract meta-features $\boldsymbol{\phi}_t \leftarrow \textsc{Extract}(W)$
    for $i = 1$ to K **do**
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \mathbf{b}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t}$
    end for
    Select detector $i_t \leftarrow \arg\max_i \text{UCB}_i$
    for $c = 1$ to C **do**
      Sample $\psi_c \sim \text{Beta}(a_{i_t,c}, b_{i_t,c})$
    end for
    Select config $c_t \leftarrow \arg\max_c \psi_c$
    $d_t \leftarrow D_{i_t}(\boldsymbol{\eta}_{i_t,c_t}).\textsc{Detect}(W)$
    if $d_t = 1$ **then**
      $\mathcal{M}_t \leftarrow \mathcal{M}_0$
    end if
    Compute reward r_t using Eq. (7)
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t \boldsymbol{\phi}_t^\top$; $\mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t \boldsymbol{\phi}_t$
    $\tilde{r}_t \leftarrow (r_t + 1)/2$
    $a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t$; $b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t)$
    $W \leftarrow \emptyset$
  end if
end for

where $\mathcal{M}_0$ represents the initial untrained state. The classifier then resumes incremental learning from subsequent instances:

$$\mathcal{M}_{t+s} \leftarrow \text{Update}(\mathcal{M}_{t+s-1}, (\mathbf{x}_{t+s}, y_{t+s})) \quad \forall s > 0 \tag{9}$$

The complete procedure is formalized in Algorithm 1.

3.4 Computational Complexity

AutoDD has per-window complexity of $O(wd + Km^2)$, where w is the AutoDD window size, d is the input feature dimensionality, K is the number of detectors, and $m = 15$ is the meta-feature count. The dominant costs are meta-feature extraction ($O(wd)$) and LinUCB matrix operations ($O(Km^2)$). Space complexity is $O(Km^2 + wd)$ for storing K covariance matrices of size $m \times m$ and the sliding window buffer.

3.5 Theoretical Analysis

We formalize AutoDD's optimization objective and provide regret guarantees for the hierarchical bandit approach.[1]

Problem Formulation. A data stream $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ exhibits concept drift when distributions change:

$$\exists t, t' : P_t(\mathbf{X}, Y) \neq P_{t'}(\mathbf{X}, Y) \tag{10}$$

[1]Complete proofs are provided in the supplementary material.

Given detector portfolio $\mathcal{D} = \{D_1, \ldots, D_K\}$ where each D_i has configuration space $\mathcal{H}_i$, AutoDD solves the streaming CASH problem [Hutter *et al.*, 2019]:

$$(D_t^*, \boldsymbol{\eta}_t^*) = \arg\max_{D_i \in \mathcal{D}, \boldsymbol{\eta} \in \mathcal{H}_i} \mathbb{E}[\mathcal{M}(D_i(\boldsymbol{\eta}), \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{11}$$

where $\boldsymbol{\phi}_t$ is the stream context. The hierarchical approach decomposes this into two sequential decisions: detector selection (Level 1) and hyperparameter configuration (Level 2).

Assumptions. We make three standard assumptions from the contextual bandit literature [Abbasi-Yadkori *et al.*, 2011; Lattimore and Szepesvári, 2020]:

- **(A1) Bounded features:** $\|\boldsymbol{\phi}_t\|_2 \leq L$ for all t, satisfied by $[0, 1]$ normalization with $L = \sqrt{m}$.
- **(A2) Linear reward:** For each detector i, there exists $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ with $\|\boldsymbol{\theta}_i^*\|_2 \leq S$ such that $\mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t] = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*$.
- **(A3) Sub-Gaussian noise:** Reward noise is σ_{noise}-sub-Gaussian, holding with $\sigma_{\text{noise}} = 1$ for bounded rewards in $[-1, 1]$.

The following result shows LinUCB learns optimal detector selection with sublinearly shrinking regret.

Theorem 1 (Level 1 Regret). Under assumptions A1–A3, with exploration parameter $\alpha_{\text{ucb}} = \sigma_{\text{noise}} \sqrt{m \log(KT/\delta_{\text{conf}})} + S$ where T is the time horizon and $\delta_{\text{conf}} \in (0, 1)$ is the confidence parameter, LinUCB detector selection achieves with probability at least $1 - \delta_{\text{conf}}$:

$$R_1(T) = \sum_{t=1}^{T} \left(\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \right) = O(\sqrt{mT \log T}) \tag{12}$$

Proof. The proof follows directly from Theorem 2 of [Abbasi-Yadkori *et al.*, 2011]. By the self-normalized martingale inequality (Theorem 1, [Abbasi-Yadkori *et al.*, 2011]), the true parameter $\boldsymbol{\theta}_i^*$ lies within the confidence ellipsoid $\mathcal{C}_{i,t} = \{\boldsymbol{\theta} : \|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_i\|_{\mathbf{A}_i} \leq \alpha_{\text{ucb}}\}$ with probability $1 - \delta_{\text{conf}}$. Under this event, UCB selection ensures $\boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \geq \max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - 2\alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}}$. Summing and applying the elliptical potential lemma [Dani *et al.*, 2008] yields $\sum_{t=1}^{T} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \leq \sqrt{2mT \log(1 + TL^2/m)}$, giving the stated bound. □

Thompson Sampling then efficiently balances exploration and exploitation of hyperparameter configurations.

Theorem 2 (Level 2 Regret). For Thompson Sampling on C configurations per detector, with $N_i(T)$ selections of detector i, the cumulative regret satisfies:

$$R_2(T) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T) \log N_i(T)}) \tag{13}$$

Proof. Conditioned on detector i being selected, hyperparameter selection reduces to a C-armed bandit with bounded rewards. We normalize $r_t \in [-1, 1]$ to $\tilde{r}_t = (r_t + 1)/2 \in [0, 1]$ and maintain Beta posteriors updated with $\tilde{r}_t$. By Theorem 1 of [Agrawal and Goyal, 2013], Thompson Sampling achieves expected regret $O(\sqrt{CN_i(T) \log N_i(T)})$ over $N_i(T)$ rounds. Since $\sum_i N_i(T) = T$, summing across detectors yields the stated bound. □

The combined regret remains sublinear, ensuring AutoDD's average per-round suboptimality vanishes over time.

Corollary (Combined Regret). Let (i^*, c^*) denote the optimal detector-configuration pair. The total regret decomposes as $R(T) = R_1(T) + R_2(T)$, where $R_1(T)$ captures suboptimality from detector selection and $R_2(T)$ from configuration selection. The combined bound is:

$$R(T) = O(\sqrt{mT \log T} + \sqrt{CKT \log T}) \tag{14}$$

Proof. By Cauchy-Schwarz, $\sum_i \sqrt{N_i(T)} \leq \sqrt{K \sum_i N_i(T)} = \sqrt{KT}$. Thus $R_2(T) = O(\sqrt{CKT \log T})$, and adding $R_1(T)$ gives the result. □

This sublinear regret guarantees convergence to optimal detector-configuration pairs:

$$\lim_{T \to \infty} \frac{R(T)}{T} = 0 \tag{15}$$

Sample Complexity. To achieve average regret ϵ, AutoDD requires $T = O\left(\frac{(m+CK)\log(K/\delta_{\text{conf}})}{\epsilon^2}\right)$ samples, following from inverting the regret bound.

4 Experiments

We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams (Table 3). Synthetic datasets are generated using River's [Montiel *et al.*, 2021] stream generators with predefined drift points exhibiting multiple drift types (sudden, gradual, incremental, and normal), while real-world datasets are standard benchmarks from the MOA framework [Bifet *et al.*, 2010] with unknown drift patterns. To demonstrate robustness, we evaluate all methods using two base classifiers: GaussianNB and Hoeffding Adaptive Tree (HAT) [Domingos and Hulten, 2000]; fixed classifiers isolate drift detection performance independently of classifier selection effects. We compare AutoDD against twelve baselines: a no-detector baseline, seven individual detectors from AutoDD's portfolio (ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN), and four recent state-of-the-art methods including MDDM [Pesaranghader *et al.*, 2017], ADDM [Mayaki and Riveill, 2022], PUDD [Lu *et al.*, 2025], and Meta-ADD [Yu *et al.*, 2021]. All detectors use recommended hyperparameter settings from the literature.

We measure prequential accuracy as the percentage of correct predictions using test-then-train evaluation. For synthetic datasets with known drift points, we report F1-score computed by matching detected drifts to ground truth using a tolerance window of $\omega_{\text{tol}} = 100$ instances following standard practice [Bifet and Gavaldà, 2007]. For real-world datasets, we report the number of detected drifts (DN). AutoDD uses all 15 meta-features with window size $w = 100$, exploration parameter $\alpha_{\text{ucb}} = 1.0$, regularization $\gamma = 1.0$, and $C = 5$ hyperparameter configurations per detector. Results are averaged over 10 runs. Implementation uses Python with River [Montiel *et al.*, 2021] on a MacBook Pro with Apple M2 Pro and 16GB RAM.[2]

[2]Code and datasets: https://anonymous.4open.science/r/AutoDD-BCCF

Table 3: Dataset characteristics (Multiple* = Sudden, Gradual, Incremental, and Normal drift types)

Dataset	Type	Instances	Features	Drift Type
SEA	Synthetic	500,000	3	Multiple*
HYP	Synthetic	500,000	10	Multiple*
AGR	Synthetic	100,000	9	Multiple*
RBF	Synthetic	200,000	10	Multiple*
RTG	Synthetic	100,000	30	Multiple*
Elec	Real-world	45,312	8	Unknown
Weather	Real-world	18,159	8	Unknown
Spam	Real-world	4,601	57	Unknown
Airlines	Real-world	539,383	7	Unknown
Poker	Real-world	829,201	10	Unknown

5 Results

We conduct comprehensive evaluation comparing AutoDD with classical detectors and state-of-the-art methods across both base classifiers.

Synthetic Datasets: Table 4 presents results on synthetic benchmarks with known drift points. AutoDD consistently achieves the highest mean accuracy across both base classifiers, demonstrating robust performance regardless of the underlying learning paradigm. With GaussianNB, AutoDD attains 81.9% mean accuracy, while with HAT it reaches 84.0%, outperforming all state-of-the-art detectors in both settings. Meta-ADD ranks as the second-best method, achieving 80.9% with GaussianNB and 81.4% with HAT. AutoDD demonstrates particular strength on datasets with abrupt drift patterns, where its ability to rapidly switch between detectors provides substantial benefits. On the SEA dataset, AutoDD achieves 93.1% accuracy with GaussianNB and 96.7% with HAT, significantly outperforming all baselines. Similar improvements are observed on the AGR dataset with 78.5% and 82.4% accuracy respectively. Meta-ADD performs competitively on gradual drift scenarios due to its learned drift-type representations. Notably, fixed detectors exhibit inconsistent performance across different drift types, with HDDM-W achieving strong results on some datasets but underperforming on others, highlighting the fundamental need for adaptive detector selection.

Real-world Datasets: Table 5 presents results on real-world benchmarks where drift patterns are unknown and potentially more complex than synthetic scenarios. AutoDD demonstrates strong generalization, achieving the highest mean accuracy with both base classifiers. With GaussianNB, AutoDD attains 79.1% mean accuracy, improving to 83.1% with HAT, consistently outperforming all state-of-the-art detectors. ADDM emerges as the second-best performer with 75.1% accuracy using GaussianNB and 77.8% with HAT. AutoDD's improvements are consistent across all five real-world datasets, demonstrating that its adaptive selection mechanism generalizes well to diverse application domains. On the Electricity dataset, AutoDD achieves 89.6% with GaussianNB and 93.8% with HAT. On Weather, it reaches 80.0% and 85.1% respectively, while on Spam it attains near-perfect accuracy of 97.9% and 99.0%. Importantly, AutoDD maintains well-balanced drift detection behavior, avoiding both over-

Table 4: Results on synthetic datasets with both base classifiers. **Bold**: best, Underline: second-best.

	GaussianNB (GNB)												Hoeffding Adaptive Tree (HAT)											
	SEA		HYP		AGR		RBF		RTG		Mean		SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	83.8	0.40	76.4	0.34	78.8	0.41	50.4	0.09	58.2	0.10	69.5	0.27	86.2	0.42	79.1	0.36	81.3	0.43	53.1	0.11	60.8	0.12	72.1	0.29
ADWIN	84.0	0.61	81.9	0.00	71.5	0.51	69.6	0.00	64.5	0.00	74.3	0.22	88.1	0.64	85.5	0.06	74.6	0.55	73.3	0.07	68.3	0.11	77.9	0.29
DDM	83.8	0.00	86.1	0.63	71.6	0.49	71.3	0.64	64.4	0.00	75.4	0.35	88.0	0.10	89.0	0.67	74.7	0.52	74.6	0.69	68.1	0.13	78.9	0.42
EDDM	84.3	0.40	85.7	0.63	65.3	0.24	67.7	0.38	65.0	0.40	73.6	0.41	88.0	0.46	88.6	0.68	69.2	0.29	71.8	0.42	68.8	0.47	77.3	0.46
HDDM-A	84.6	0.69	81.5	0.00	72.2	0.63	69.7	0.00	64.4	0.00	74.5	0.26	88.3	0.64	86.8	0.06	75.1	0.63	73.8	0.07	69.1	0.11	78.6	0.30
HDDM-W	84.4	0.66	86.6	0.71	73.1	0.67	70.1	0.51	65.8	0.56	76.0	0.62	88.4	0.65	89.5	0.72	75.9	0.67	73.9	0.53	69.7	0.58	79.5	0.63
Page-Hinkley	84.7	0.70	81.5	0.00	71.5	0.49	69.4	0.00	64.5	0.00	74.3	0.24	88.2	0.65	86.0	0.06	74.8	0.52	73.5	0.07	68.5	0.11	78.2	0.28
KSWIN	83.7	0.35	85.7	0.65	71.5	0.50	70.0	0.61	65.2	0.39	75.2	0.50	88.0	0.42	88.3	0.66	74.5	0.53	73.7	0.63	68.9	0.43	78.7	0.53
MDDM	86.7	0.65	87.2	0.68	72.3	0.71	69.8	0.56	66.1	0.56	76.4	0.63	89.1	0.67	90.3	0.75	75.5	0.75	73.8	0.60	69.8	0.61	78.4	0.68
ADDM	87.7	0.83	88.7	0.73	73.6	0.67	70.9	0.62	67.6	0.71	77.7	0.71	89.8	0.86	91.9	0.77	76.6	0.72	75.0	0.69	71.3	0.77	79.3	0.76
PUDD	86.2	0.81	89.5	0.84	73.7	0.72	73.4	0.72	67.8	0.71	78.1	0.76	89.8	0.83	92.6	0.83	77.1	0.76	76.4	0.74	71.9	0.74	79.7	0.78
Meta-ADD	88.9	0.87	90.8	**0.91**	74.1	0.79	**78.0**	0.70	**72.9**	0.76	80.9	0.81	90.9	0.91	92.5	0.92	77.1	0.81	**84.8**	0.74	**80.4**	0.79	81.4	0.83
AutoDD	**93.1**	**0.90**	**94.6**	**0.91**	**78.5**	**0.84**	74.3	**0.81**	69.2	**0.84**	**81.9**	**0.86**	**96.7**	**0.93**	**98.4**	**0.93**	**82.4**	**0.90**	75.8	**0.84**	70.9	**0.88**	**84.0**	**0.87**

detection and under-detection patterns that plague fixed detectors. On the Airlines dataset, AutoDD detects 118 drifts, compared to HDDM-W's excessive 269 detections and ADWIN's insufficient 76 detections, demonstrating appropriate sensitivity calibration.

Statistical Significance: To validate the observed performance differences, we conduct statistical significance analysis using GaussianNB as the base classifier. Figure 4 presents the Critical Difference diagram based on the Friedman test with Nemenyi post-hoc analysis at significance level $p = 0.05$. AutoDD achieves the best average rank among all methods, demonstrating statistically significant improvements over the baseline detectors. The state-of-the-art methods Meta-ADD, ADDM, PUDD, and MDDM follow in subsequent ranks, while portfolio detectors cluster in middle positions. The No Detector baseline performs worst, confirming that drift detection is essential for streaming data. Connecting bars in the diagram indicate groups of methods that are not statistically different from each other, highlighting that AutoDD's improvements are significant rather than due to random variation.

Ablation Study: To understand the contribution of each component, we systematically remove AutoDD components and evaluate the impact on performance using GaussianNB as the base classifier. Table 6 presents results averaged across all ten datasets. The hierarchical selection mechanism proves critical for AutoDD's performance. Removing Level 1 LinUCB detector selection and replacing it with random selection causes the largest performance drop, reducing accuracy from 80.5% to 62.5%, an 18.0% decrease. This substantial degradation confirms that intelligent detector-stream matching is essential for effective drift detection. Removing Level 2 Thompson Sampling hyperparameter configuration reduces accuracy to 75.3%, demonstrating that adaptive tuning contributes meaningfully beyond fixed default configurations. All three meta-feature categories contribute substantially to the overall performance. Statistical features have the largest impact, with their removal reducing accuracy by 9.4%. Stationarity features contribute 7.1% to performance, while temporal features account for 6.4%. Figure 5 visualizes AutoDD's adaptive behavior on the Hyperplane dataset, showing how the framework dynamically switches between detectors in response to changing stream conditions and drift events.

Sensitivity Study: We analyze AutoDD's sensitivity to key hyperparameters using GaussianNB as the base classifier (Figure 6). The optimal window size of $w = 100$ achieves 92.6% selection accuracy, balancing statistical stability with drift responsiveness. Smaller windows suffer from high variance, while larger windows cause delayed drift response. For the exploration parameter, $\alpha_{\text{ucb}} = 1.0$ achieves best performance, with lower values causing exploitation bias and higher values leading to excessive exploration. The regularization parameter γ shows robust performance across the range from 0.5 to 2.0, with $\gamma = 1.0$ being optimal. This stability indicates that AutoDD is not overly sensitive to hyperparameter choices around the recommended defaults.

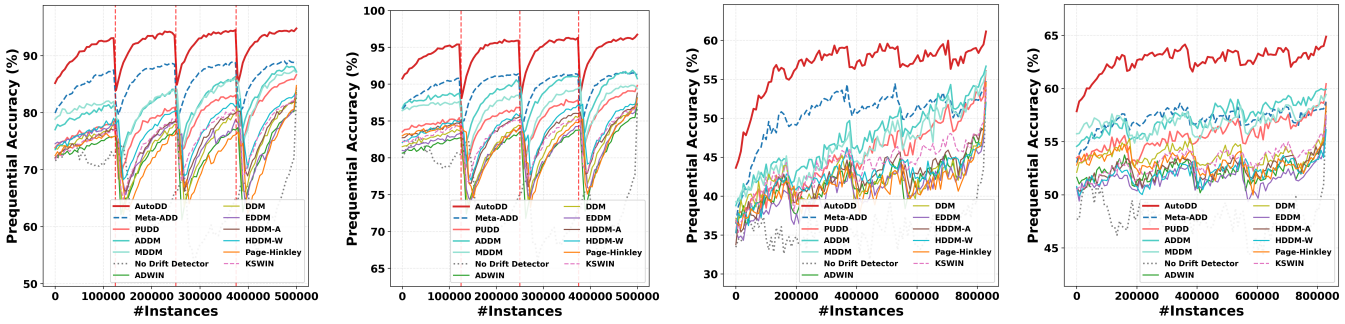
Computational Efficiency: For practical deployment in real-time streaming applications, computational efficiency is crucial. Figure 7 compares runtime and memory usage across all methods using GaussianNB as the base classifier. AutoDD achieves strong computational efficiency through its incremental meta-feature computation using online statistics, processing instances at 0.29 ms per instance. This is substantially faster than Meta-ADD at 1.25 ms per instance, making AutoDD 4.3 times more efficient while simultaneously achieving higher accuracy. Memory usage is 7.5 MB for AutoDD, which is modest compared to Meta-ADD's 47.7 MB neural network overhead. Among the state-of-the-art methods, AutoDD is faster than both PUDD at 0.33 ms and ADDM at 0.60 ms per instance. While classical detectors like MDDM are faster at 0.12 ms per instance, AutoDD delivers substantially higher accuracy with only modest additional overhead. This efficiency-accuracy trade-off makes AutoDD practical for deployment in high-velocity streaming scenarios where both performance and responsiveness are important.

6 Conclusion

We propose AutoDD, a novel framework that automatically selects and configures drift detectors for data streams. The framework extracts comprehensive meta-features and employs a two-level hierarchical selection strategy using LinUCB for detector selection and Thompson Sampling for hyperparameter configuration. Theoretical analysis provides sublinear regret bounds guaranteeing convergence to optimal detector-configuration pairs. Extensive experimental results on synthetic and real-world datasets demonstrate that Au-

Table 5: Results on real-world datasets with both base classifiers. DN = detected drifts. **Bold**: best, Underline: second-best.

	GaussianNB (GNB)												Hoeffding Adaptive Tree (HAT)											
	Elec		Weather		Spam		Airlines		Poker		Mean		Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	73.2	0	68.9	0	77.2	0	59.3	0	54.6	0	66.6	0	76.5	0	71.4	0	80.1	0	62.1	0	56.9	0	69.4	0
ADWIN	79.2	22	69.3	7	92.1	2	62.5	76	54.3	3	71.5	22.0	82.4	22	73.1	9	94.6	2	66.3	76	57.7	3	74.8	22.4
DDM	82.3	22	70.6	5	91.8	2	61.8	20	54.2	3	72.1	10.4	85.5	22	73.5	7	93.9	2	65.2	20	58.0	3	75.2	10.8
EDDM	82.4	22	73.8	9	92.9	2	61.3	182	53.6	4	72.8	43.8	85.2	22	77.1	9	94.8	2	64.6	182	57.8	4	75.9	43.8
HDDM-A	82.7	22	72.2	9	92.5	2	62.6	114	54.6	6	72.9	30.6	86.3	22	75.8	9	94.7	2	66.1	114	58.5	6	76.3	30.6
HDDM-W	81.9	22	72.6	9	91.9	1	61.0	269	52.3	394	71.9	139.0	85.1	22	76.0	9	94.5	1	64.9	269	56.8	404	75.5	141.0
Page-Hinkley	78.0	16	69.4	6	91.8	1	62.8	64	54.3	17	71.3	20.8	81.7	16	72.8	6	94.2	1	66.5	64	57.5	17	74.5	20.8
KSWIN	81.1	22	68.0	9	92.6	2	61.6	269	53.7	266	71.4	113.6	84.3	22	71.2	9	94.9	2	65.4	269	57.0	266	74.6	113.6
MDDM	83.9	22	74.8	9	92.8	2	62.2	110	56.1	190	74.0	66.6	86.5	22	78.0	9	94.6	2	66.0	110	60.0	190	77.0	66.6
ADDM	84.8	22	76.3	9	93.9	2	63.9	74	56.5	177	75.1	56.8	87.3	22	79.2	9	95.8	2	66.9	74	59.6	177	77.8	56.8
PUDD	84.4	22	75.4	9	93.2	2	64.0	117	56.2	81	74.6	46.2	87.5	22	78.6	9	95.6	2	67.7	117	59.7	81	77.8	46.2
Meta-ADD	85.4	22	74.9	9	94.5	2	63.8	139	55.5	182	74.8	70.8	87.9	22	78.2	9	95.9	2	66.5	139	59.6	182	77.6	70.8
AutoDD	**89.6**	22	**80.0**	9	**97.9**	2	**68.2**	118	**59.7**	188	**79.1**	67.8	**93.8**	22	**85.1**	9	**99.0**	2	**72.6**	118	**65.1**	188	**83.1**	67.8



(a) SEA (GNB) (b) SEA (HAT) (c) Poker (GNB) (d) Poker (HAT)

Figure 3: Prequential accuracy over time. SEA (synthetic) and Poker (real-world) datasets with GaussianNB and HAT base classifiers.

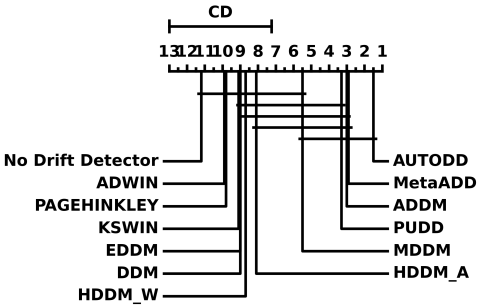


Figure 4: Critical Difference diagram with connecting bars indicating statistically equivalent methods.

Table 6: Ablation study results (all 10 datasets)

Configuration	Accuracy	F1	ΔAcc
Full AutoDD	**80.5%**	**0.64**	–
w/o Temporal	74.2%	0.59	-6.4%
w/o Statistical	71.2%	0.57	-9.4%
w/o Stationarity	73.5%	0.59	-7.1%
w/o LinUCB (Level 1)	62.5%	0.50	-18.0%
w/o Thompson (Level 2)	75.3%	0.60	-5.2%

toDD achieves significant improvements over fixed detector

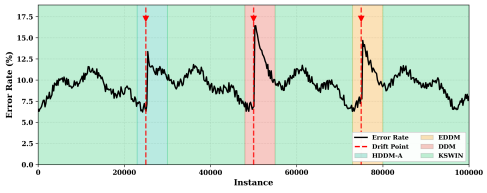
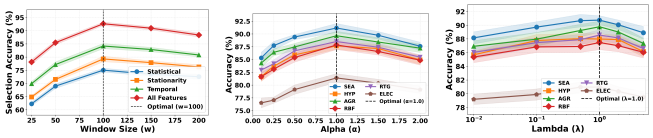


Figure 5: Detector selection on Hyperplane stream showing active detectors, error rate, and drift points.



(a) Window size w (b) Exploration α_{ucb} (c) Regularization γ

Figure 6: Hyperparameter sensitivity analysis showing selection accuracy across different values of (a) window size, (b) exploration parameter, and (c) regularization parameter.

baselines and state-of-the-art methods while requiring no expert intervention. Future work includes incorporating multi-objective optimization to balance accuracy, detection latency, and computational efficiency. We also plan to expand AutoDD to different types of data stream problems, including unsupervised drift detection scenarios where labels are un-

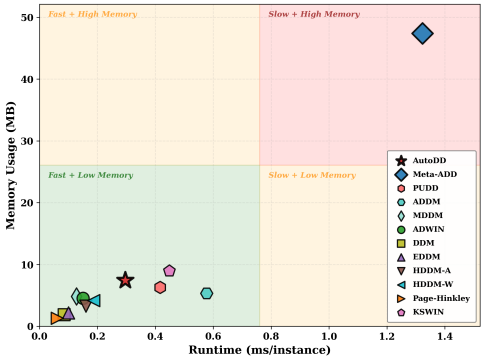


Figure 7: Runtime vs. memory trade-off.

available. Additionally, exploring better adaptation strategies beyond model reset, such as incremental learning and ensemble updating, represents a promising direction for further improving post-drift recovery.

References

[Abbasi-Yadkori *et al.*, 2011] Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Improved algorithms for linear stochastic bandits. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2312–2320, 2011.

[Agrawal and Goyal, 2012] Shipra Agrawal and Navin Goyal. Analysis of Thompson sampling for the multi-armed bandit problem. In *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*, pages 39.1–39.26. PMLR, 2012.

[Agrawal and Goyal, 2013] Shipra Agrawal and Navin Goyal. Further optimal regret bounds for Thompson sampling. In *Proceedings of the 16th International Conference on Artificial Intelligence and Statistics*, pages 99–107. PMLR, 2013.

[ao Gama *et al.*, 2014] Jo ao Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4):1–37, 2014.

[Baena-García *et al.*, 2006] Manuel Baena-García, José del Campo-Ávila, Raúl Fidalgo, Albert Bifet, R. Gavaldà, and R. Morales-Bueno. Early drift detection method. In *Fourth International Workshop on Knowledge Discovery from Data Streams*, volume 6, pages 77–86, 2006.

[Bifet and Gavaldà, 2007] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. SIAM, 2007.

[Bifet *et al.*, 2010] Albert Bifet, Geoff Holmes, Bernhard Pfahringer, Philipp Kranen, Hardy Kremer, Timm Jansen, and Thomas Seidl. MOA: Massive online analysis, a framework for stream classification and clustering. In *Proceedings of the First Workshop on Applications of Pattern Analysis*, pages 44–50. PMLR, 2010.

[Celik *et al.*, 2023] Bilge Celik, Prabhant Singh, and Joaquin Vanschoren. Online AutoML: An adaptive AutoML framework for online learning. *Machine Learning*, 112(6):1897–1921, 2023.

[Dani *et al.*, 2008] Varsha Dani, Thomas P Hayes, and Sham M Kakade. Stochastic linear optimization under bandit feedback. In *Proceedings of the 21st Annual Conference on Learning Theory*, pages 355–366, 2008.

[Dasu *et al.*, 2006] Tamraparni Dasu, Shankar Krishnan, Suresh Venkatasubramanian, and Ke Yi. An information-theoretic approach to detecting changes in multidimensional data streams. In *Proceedings of Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006.

[Domingos and Hulten, 2000] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 71–80. ACM Press, 2000.

[Friás-Blanco *et al.*, 2015] Isvani Friás-Blanco, J. de Campo-Ávila, G. Ramos-Jiménez, Rafael Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.

[Gama *et al.*, 2004] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Brazilian Symposium on Artificial Intelligence*, pages 286–295. Springer, 2004.

[Hutter *et al.*, 2019] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. Automated machine learning: Methods, systems, challenges. In *Automated Machine Learning*. Springer, 2019.

[Kifer *et al.*, 2004] Daniel Kifer, Shai Ben-David, and Johannes Gehrke. Detecting change in data streams. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, volume 4, pages 180–191, 2004.

[Lattimore and Szepesvári, 2020] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.

[Li *et al.*, 2010] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670. ACM, 2010.

[Losing *et al.*, 2018] Viktor Losing, Barbara Hammer, and Heiko Wersing. Incremental on-line learning: A review and comparison of state of the art algorithms. *Neurocomputing*, 275:1261–1274, 2018.

[Lu *et al.*, 2014] Ning Lu, Guangquan Zhang, and Jie Lu. Concept drift detection via competence models. *Artificial Intelligence*, 209:11–28, 2014.

[Lu *et al.*, 2019] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang. Learning under concept drift: A review.

IEEE Transactions on Knowledge and Data Engineering, 31(12):2346–2363, 2019.

[Lu *et al.*, 2025] Pengqian Lu, Jie Lu, Anjin Liu, and Guangquan Zhang. Early concept drift detection via prediction uncertainty. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.

[Mayaki and Riveill, 2022] Mansour Zoubeirou A Mayaki and Michel Riveill. Autoregressive based drift detection method. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2022.

[Montiel *et al.*, 2021] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. River: Machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

[Page, 1954] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[Pesaranghader *et al.*, 2017] Ali Pesaranghader, Herna Viktor, and Eric Paquet. McDiarmid drift detection methods for evolving data streams. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1663–1670. IEEE, 2017.

[Raab *et al.*, 2020] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351, 2020.

[Tsay, 1989] Ruey S. Tsay. Testing and modeling threshold autoregressive processes. *Journal of the American Statistical Association*, 84:231–240, 1989.

[Verma *et al.*, 2024] Nilesh Verma, Albert Bifet, Bernhard Pfahringer, and Maroua Bahri. ASML: A scalable and efficient AutoML solution for data streams. In *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, Proceedings of Machine Learning Research. PMLR, 2024.

[Yu *et al.*, 2021] Hang Yu, Jie Lu, and Guangquan Zhang. Meta-ADD: A meta-learning based pre-trained model for concept drift active detection. *Information Sciences*, 578:1–14, 2021.